

Experiment Report

Student: Denisenko Sofiia 狄苏菲

Student id: 1820249051

Course: Big Data Analysis Technology

Experiment Topic: Data Visualization

Questions

1. Explain 4 types of data analysis?

1.1 Descriptive analysis

Deals with what happened in the Past.

This type of analysis focuses on summarizing and describing the characteristics of the data. It helps understand the basic patterns, trends, and distributions within the dataset.

1.2 Diagnostic analysis

Deals with why it happened in the Past.

This analysis delves deeper than descriptive analysis. It aims to uncover the root causes behind the patterns and trends that were observed in the descriptive analysis. It helps to understand why things happened the way they did.

1.3 Predictive analysis

Deals with what will happen in the Future.

This type of analysis uses historical data to build models that predict future outcomes. It involves identifying patterns and relationships in the data and using them to forecast what might happen in the future.

1.4 Perspective analysis

Deals with what step to take to avoid a problem in the Future.

This is the most advanced form of analysis. It goes beyond prediction by recommending specific actions to take based on the insights gained from the analysis. It helps to make decisions that can positively influence future outcomes.

2. Explain Data Visualization tools?

- **Microsoft Power BI**

Provides real-time data insights through dashboards.

Key Features: Data Connectivity, User Friendly, Data Transformation with Power Query, Updates Data Automatically

- **Tableau**

A tool for creating interactive data visualizations.

Key Features: Dynamic Dashboards, Enhanced Data Reliability, Data Exploration

- **D3.js (Data-Driven Documents)**

An open-source library written in javascript creates dashboards in web browsers.

Key Features: Data Binding, Highly Customizable, Data Transformation and Manipulation Interactivity

- **Plotly**

A graphing library used for creating interactive, web-based data visualizations, including charts, graphs, and dashboards.

Key Features: Interactive Visualizations, Data Integration, Customizable Dashboards, Scalability and Collaboration

- **Matplotlib & Seaborn**

Matplotlib is a Python library for creating visualizations. Seaborn is a Python data visualization library built on top of Matplotlib. It also makes visualizations.

Key Features:

Matplotlib: Customization, Static Plotting, Subplots & Layout

Seaborn: Statistical Plots Ease of use Aesthetics Integration

3. What kind of Data Visualization tools can support which type of data analysis (in question 1)?

3.1 Descriptive analysis

- **Tableau.** Excellent for creating interactive dashboards, exploring data visually, and uncovering patterns in distributions.
- **Power BI.** Great for creating interactive dashboards that summarize data with charts, graphs, and tables.

- **Matplotlib & Seaborn.** The foundation of Python visualization. Matplotlib provides the basic plotting tools, while Seaborn offers higher-level and more visually appealing visualizations.

3.2 Diagnostic analysis

- **Tableau.** Strong tools for interactive filtering, drilling down, and creating cross-tabulations to analyze relationships.
- **Power BI.** Offers filtering, drilling down, and interactive features to help identify potential causes of trends.
- **D3.js.** Can create sophisticated visualizations that facilitate interactive exploration of relationships and potential causes.

3.3 Predictive analysis

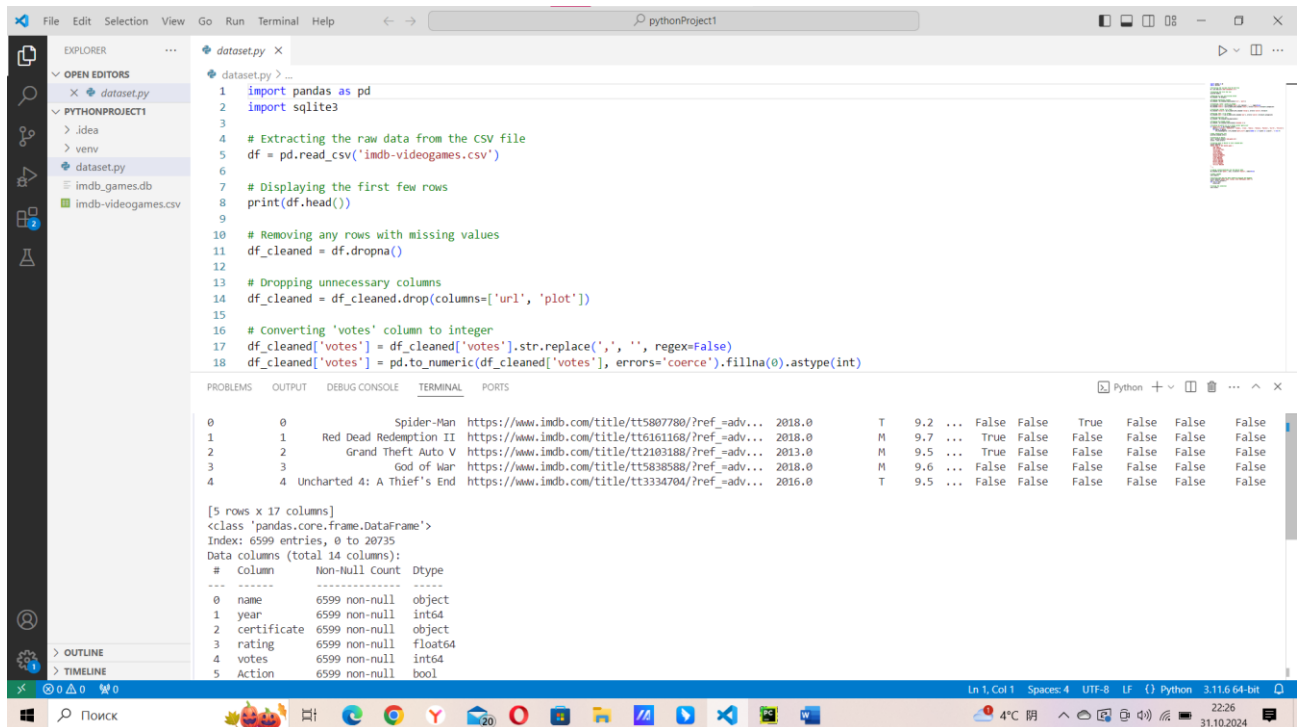
- **Plotly.** Excellent for visualizing predictions, particularly for time series forecasting. You can create interactive charts showing actual data and predictions.
- **D3.js.** Provides the flexibility to create custom visualizations to display predictions effectively.

3.4 Perspective analysis

- **D3.js.** Offers the most customizability for building visualizations that support decision-making processes.

Preparations to the Experiment

1. I did the experiment in VS code. Installing pandas and running code in file *“dataset.py”*.



The screenshot shows the VS Code editor with the file `dataset.py` open. The code in the file is as follows:

```
1 import pandas as pd
2 import sqlite3
3
4 # Extracting the raw data from the CSV file
5 df = pd.read_csv('imdb-videogames.csv')
6
7 # Displaying the first few rows
8 print(df.head())
9
10 # Removing any rows with missing values
11 df_cleaned = df.dropna()
12
13 # Dropping unnecessary columns
14 df_cleaned = df_cleaned.drop(columns=['url', 'plot'])
15
16 # Converting 'votes' column to integer
17 df_cleaned['votes'] = df_cleaned['votes'].str.replace(',', '', regex=False)
18 df_cleaned['votes'] = pd.to_numeric(df_cleaned['votes'], errors='coerce').fillna(0).astype(int)
```

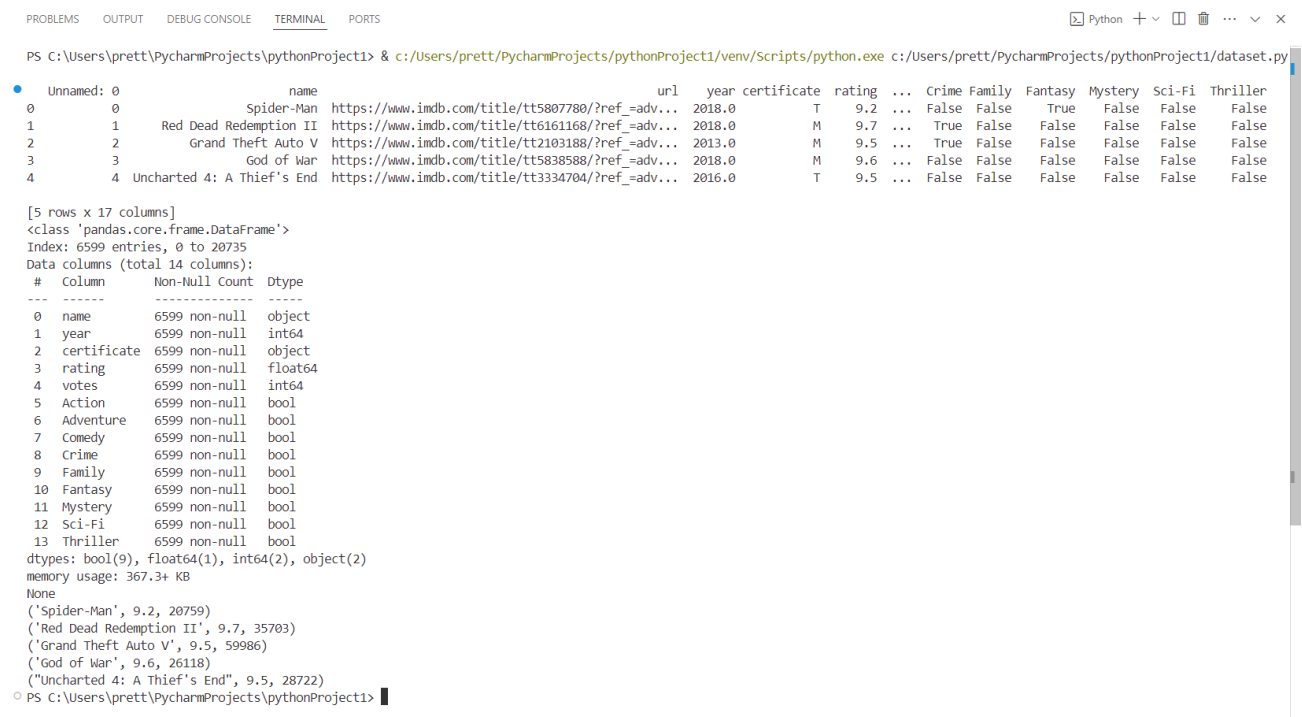
The terminal output shows the first 5 rows of the dataset:

0	0	Spider-Man	https://www.imdb.com/title/tt5807780/?ref=adv...	2018.0	T	9.2	...	False	False	True	False	False	False	False	False	False	False
1	1	Red Dead Redemption II	https://www.imdb.com/title/tt6161168/?ref=adv...	2018.0	M	9.7	...	True	False	False	False	False	False	False	False	False	False
2	2	Grand Theft Auto V	https://www.imdb.com/title/tt2103188/?ref=adv...	2013.0	M	9.5	...	True	False	False	False	False	False	False	False	False	False
3	3	God of War	https://www.imdb.com/title/tt5838588/?ref=adv...	2018.0	M	9.6	...	False	False	False	False	False	False	False	False	False	False
4	4	Uncharted 4: A Thief's End	https://www.imdb.com/title/tt3334704/?ref=adv...	2016.0	T	9.5	...	False	False	False	False	False	False	False	False	False	False

The terminal also shows the following information:

```
[5 rows x 17 columns]
<class 'pandas.core.frame.DataFrame'>
Index: 6599 entries, 0 to 20735
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   name        6599 non-null   object
1   year        6599 non-null   int64
2   certificate  6599 non-null   object
3   rating      6599 non-null   float64
4   votes       6599 non-null   int64
5   Action      6599 non-null   bool
```

Results:



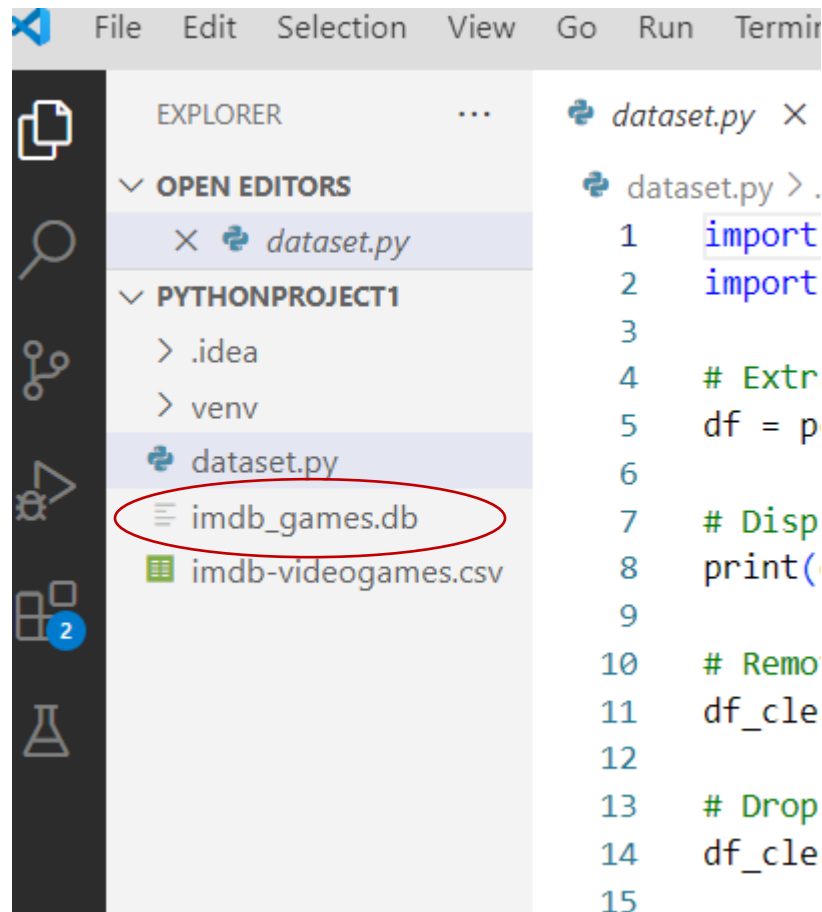
The terminal output shows the first 5 rows of the dataset:

0	0	Spider-Man	https://www.imdb.com/title/tt5807780/?ref=adv...	2018.0	T	9.2	...	False	False	True	False	False	False	False	False	False	False
1	1	Red Dead Redemption II	https://www.imdb.com/title/tt6161168/?ref=adv...	2018.0	M	9.7	...	True	False	False	False	False	False	False	False	False	False
2	2	Grand Theft Auto V	https://www.imdb.com/title/tt2103188/?ref=adv...	2013.0	M	9.5	...	True	False	False	False	False	False	False	False	False	False
3	3	God of War	https://www.imdb.com/title/tt5838588/?ref=adv...	2018.0	M	9.6	...	False	False	False	False	False	False	False	False	False	False
4	4	Uncharted 4: A Thief's End	https://www.imdb.com/title/tt3334704/?ref=adv...	2016.0	T	9.5	...	False	False	False	False	False	False	False	False	False	False

The terminal also shows the following information:

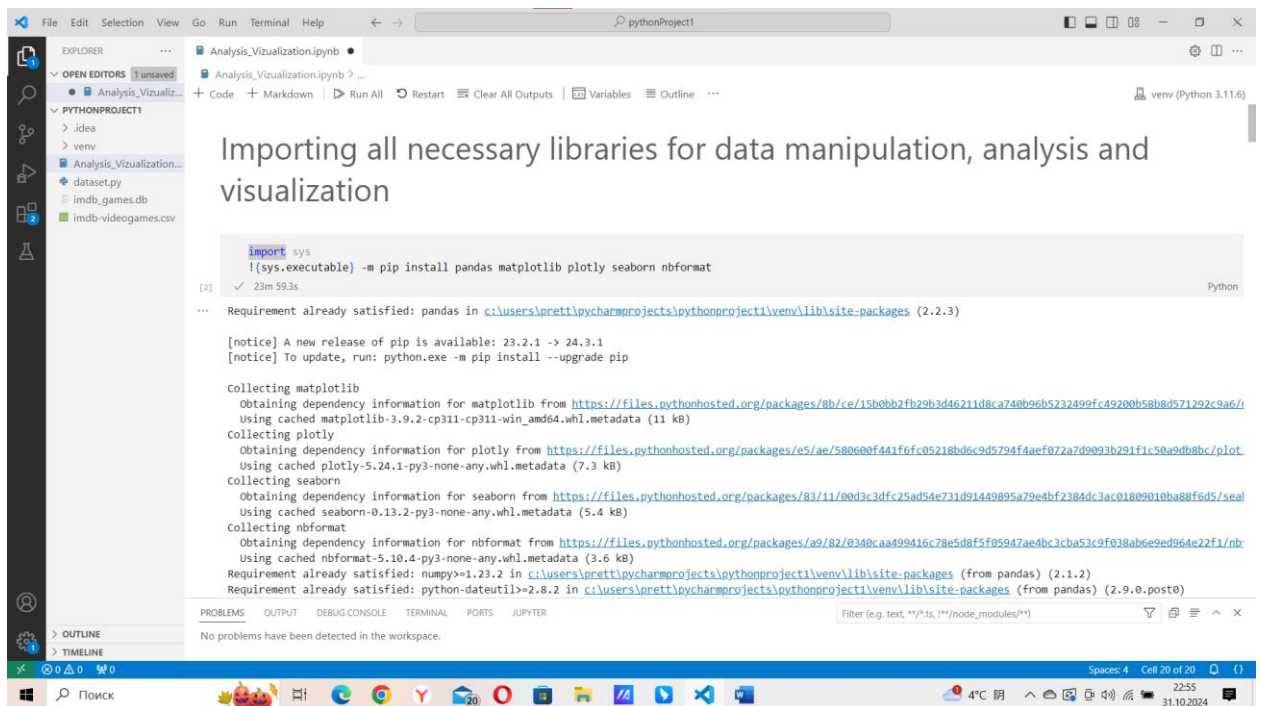
```
[5 rows x 17 columns]
<class 'pandas.core.frame.DataFrame'>
Index: 6599 entries, 0 to 20735
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   name        6599 non-null   object
1   year        6599 non-null   int64
2   certificate  6599 non-null   object
3   rating      6599 non-null   float64
4   votes       6599 non-null   int64
5   Action      6599 non-null   bool
6   Adventure   6599 non-null   bool
7   Comedy      6599 non-null   bool
8   Crime       6599 non-null   bool
9   Family      6599 non-null   bool
10  Fantasy     6599 non-null   bool
11  Mystery     6599 non-null   bool
12  Sci-Fi      6599 non-null   bool
13  Thriller    6599 non-null   bool
dtypes: bool(9), float64(1), int64(2), object(2)
memory usage: 367.3+ KB
None
('Spider-Man', 9.2, 20759)
('Red Dead Redemption II', 9.7, 35703)
('Grand Theft Auto V', 9.5, 59986)
('God of War', 9.6, 26118)
('Uncharted 4: A Thief's End', 9.5, 28722)
PS C:\Users\prett\PycharmProjects\pythonProject1>
```

Now we have a new file *“imdb_games.db”*.



2. Jupyter environment set up

Firstly, I installed all the libraries by running the first cell of “*Analysis_Vizualization.ipynb*”.



```
Collecting plotly
  Obtaining dependency information for plotly from https://files.pythonhosted.org/packages/e5/ae/580600f441f6fc05218bd6c9d5794faef072a7d9093b291f1c50a9db8bc/plotly-5.24.1-py3-none-any.whl.metadata (7.3 kB)
Using cached plotly-5.24.1-py3-none-any.whl.metadata (7.3 kB)
Collecting seaborn
  Obtaining dependency information for seaborn from https://files.pythonhosted.org/packages/83/11/00d3c3dfc25ad54e731d91449895a79e4bf2384dc3ac01809010ba88f6d5/seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Using cached seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Collecting nbformat
  Obtaining dependency information for nbformat from https://files.pythonhosted.org/packages/a9/82/0340caa499416c78e5d8f5f05947ae4bc3ba53c9f038ab6e9ed964e22f1/nbformat-5.10.4-py3-none-any.whl.metadata (3.6 kB)
Using cached nbformat-5.10.4-py3-none-any.whl.metadata (3.6 kB)
Requirement already satisfied: numpy>=1.23.2 in c:\users\prett\pycharmprojects\pythonproject1\venv\lib\site-packages (from pandas) (2.1.2)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\prett\pycharmprojects\pythonproject1\venv\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\prett\pycharmprojects\pythonproject1\venv\lib\site-packages (from pandas) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\prett\pycharmprojects\pythonproject1\venv\lib\site-packages (from pandas) (2024.2)
Collecting contourpy>=1.0.1 (from matplotlib)
  Obtaining dependency information for contourpy>=1.0.1 from https://files.pythonhosted.org/packages/8d/2f/804f02ff30a7fae21f98198828d0857439ec4c91a96e20cf2d6c493/
Using cached contourpy-1.3.0-cp311-cp311-win_amd64.whl.metadata (5.4 kB)
Collecting cycler>=0.10 (from matplotlib)
  Obtaining dependency information for cycler>=0.10 from https://files.pythonhosted.org/packages/e7/05/c19819d5e3d95294a6f5947fb9b9629efb316b96de511b418c53d245aae/
Using cached cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib)
  Obtaining dependency information for fonttools>=4.22.0 from https://files.pythonhosted.org/packages/63/f1/3a081cd047d83b5966cb0d7ef3fea929ee6eddeb94d8fbfdb2a19b/
...
----- -- 204.8/218.5 kB 12.2 kB/s eta 0:00:02
----- 218.5/218.5 kB 12.7 kB/s eta 0:00:00
Installing collected packages: fastjsonschema, tenacity, rpd-py, pyparsing, pillow, kiwisolver, fonttools, cycler, contourpy, attrs, referencing, plotly, matplotlib
Successfully installed attrs-24.2.0 contourpy-1.3.0 cycler-0.12.1 fastjsonschema-2.20.0 fonttools-4.54.1 jsonschema-4.23.0 jsonschema-specifications-2024.10.1 kiw...
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

Experiment

Running all the cells. Here you can see screenshots of the successful runs.

```
import plotly.express as px
import plotly.graph_objects as go
import pandas as pd
import sqlite3
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

[3] ✓ 1.3s

Creating a connection to the database and load the data

```
conn = sqlite3.connect('imdb_games.db')
df_cleaned = pd.read_sql_query("SELECT * FROM games", conn)
```

[4] ✓ 0.0s

Reading the “*imdb_games.db*”.

Descriptive Statistics

```
summary_stats = df_cleaned.describe()
print(summary_stats)
```

[5] ✓ 0.0s

```

...
      year      rating      votes      Action      Adventure \
count  6599.000000  6599.000000  6599.000000  6599.000000  6599.000000
mean   2006.245492   7.230732  1045.106228   0.778451   0.633429
std     9.116140    1.161120  3517.714678   0.415320   0.481904
min    1972.000000   1.000000   5.000000   0.000000   0.000000
25%    2000.000000   6.600000  29.000000   1.000000   0.000000
50%    2006.000000   7.400000  101.000000   1.000000   1.000000
75%    2014.000000   8.000000  555.500000   1.000000   1.000000
max    2022.000000   9.700000 60597.000000   1.000000   1.000000

      Comedy      Crime      Family      Fantasy      Mystery \
count  6599.000000  6599.000000  6599.000000  6599.000000  6599.000000
mean    0.117290    0.077133    0.157145    0.263525    0.062434
std     0.321791    0.266822    0.363965    0.440578    0.241960
min     0.000000    0.000000    0.000000    0.000000    0.000000
25%     0.000000    0.000000    0.000000    0.000000    0.000000
50%     0.000000    0.000000    0.000000    0.000000    0.000000
75%     0.000000    0.000000    0.000000    1.000000    0.000000
max     1.000000    1.000000    1.000000    1.000000    1.000000

      Sci-Fi      Thriller
count  6599.000000  6599.000000
mean    0.153357    0.035915
std     0.360358    0.186091
min     0.000000    0.000000
25%     0.000000    0.000000
50%     0.000000    0.000000
75%     0.000000    0.000000
max     1.000000    1.000000

```

Here we can see descriptive statistics for each numerical column.

Statistical Analysis

Top 11 games by ratings

```

conn = sqlite3.connect('imdb_games.db')

query = '''
SELECT name, rating
FROM games
ORDER BY rating DESC
LIMIT 11
'''

top_games = pd.read_sql(query, conn)

conn.close()

print(top_games.to_string(index=False))

```

[7] ✓ 0.0s

The top games of the dataset.

```
...
                                name  rating
      Red Dead Redemption II      9.7
      The Last of Us              9.7
      The Witcher 3: Wild Hunt     9.7
      Mass Effect: Legendary Edition 9.7
      The Witcher 3: Wild Hunt - Blood and Wine 9.7
      The Last of Us              9.7
      The Witcher 3: Wild Hunt - Blood and Wine 9.7
      Red Dead Redemption II      9.7
      The Witcher 3: Wild Hunt     9.7
      The Last of Us              9.7
      Mass Effect: Legendary Edition 9.7
```

Total votes for each genre

```
▷ ~
# Defining the genre columns
genre_columns = ['Action', 'Adventure', 'Comedy', 'Crime', 'Family',
                 'Fantasy', 'Mystery', 'Sci-Fi', 'Thriller']

melted_df = df_cleaned.melt(id_vars=['votes'], value_vars=genre_columns,
                            var_name='Genre', value_name='Is_Genre')

# Filtering only applicable genres
filtered_df = melted_df[melted_df['Is_Genre'] == 1]

# Group by Genre and sum votes
total_votes_by_genre = filtered_df.groupby('Genre')['votes'].sum().reset_index()

# Sort by total votes in descending order
total_votes_by_genre = total_votes_by_genre.sort_values(by='votes', ascending=False)

print(total_votes_by_genre)
```

[8] ✓ 0.0s

Total votes each genre. The “Action” is the most famous genre.

```
...
      Genre  votes
0    Action 6348811
1  Adventure 4977777
5    Fantasy 1631520
3     Crime 1343081
7    Sci-Fi  982141
2     Comedy 457004
8   Thriller 440125
6     Mystery 429245
4     Family 368054
```

Distribution of Ratings

```
mean_rating = df_cleaned['rating'].mean()

fig = px.histogram(df_cleaned, x='rating', nbins=90,
                  title='Distribution of Video Game Ratings',
                  labels={'rating': 'Rating'},
                  color_discrete_sequence=['purple'])

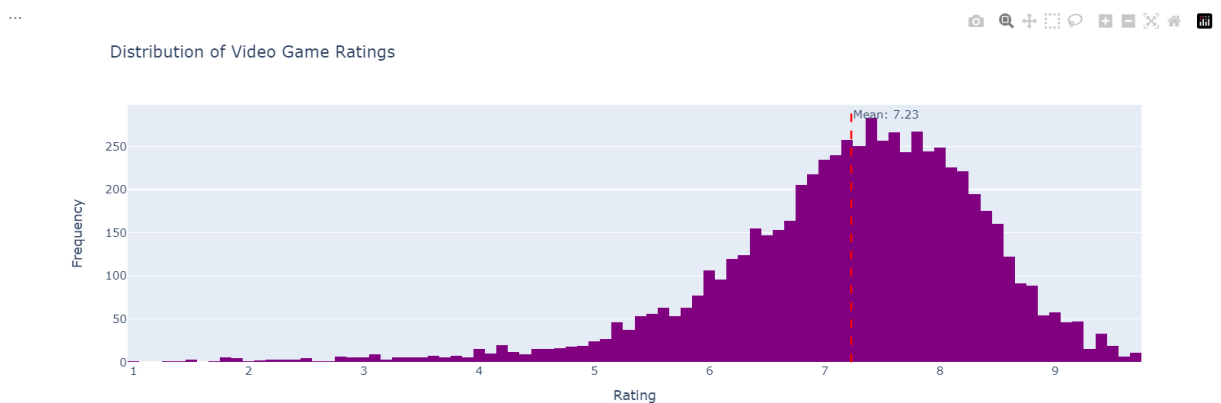
fig.add_vline(x=mean_rating, line_color='red', line_dash='dash',
              annotation_text=f'Mean: {mean_rating:.2f}',
              annotation_position='top right')

fig.update_layout(xaxis_title='Rating', yaxis_title='Frequency')

fig.show()
```

[9] ✓ 0.7s

Resulting graph:



Here we can see that there are more games with lower ratings and fewer games with very high ratings. The majority of games fall between 6 and 8 and the mean is 7,23.

Ratings Over the Years with Trendline

```
average_ratings = df_cleaned.groupby('year')['rating'].mean().reset_index()

fig = px.line(average_ratings, x='year', y='rating',
              title='Average Ratings of Video Games Over the Years',
              labels={'year': 'Year', 'rating': 'Average Rating'},
              markers=True)

trendline = go.Scatter(x=average_ratings['year'], y=average_ratings['rating'],
                      mode='lines',
                      name='Trendline',
                      line=dict(color='red'))

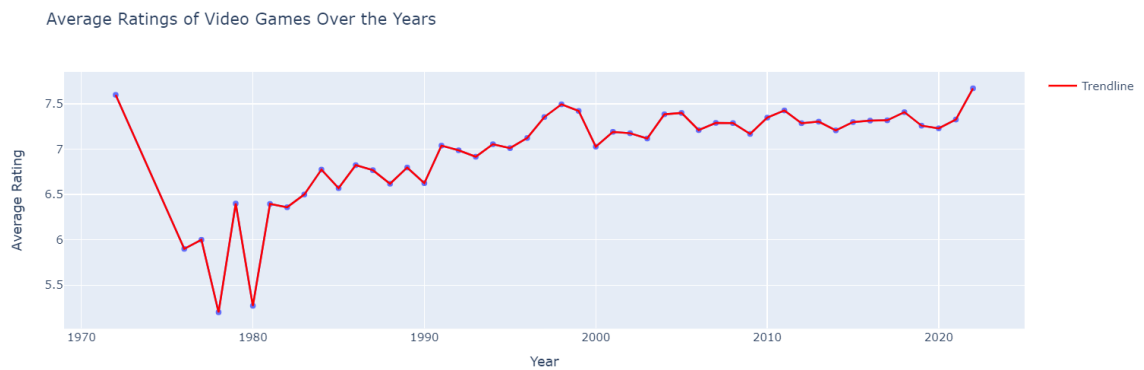
fig.add_trace(trendline)

fig.show()
```

[10] ✓ 0.0s

Python

Resulting graph:



The average ratings start very high in the early 1970s, then drop significantly. From the late 1980s to the early 2010s, the average rating stabilizes around a range of 6.5 to 7.5, indicating a more mature video game industry with consistent quality. In recent years, the average rating has increased again, reaching its highest point in the 2020s.

How are certificates related to different genres?

```
melted_df = df_cleaned.melt(id_vars=['certificate', 'rating'], value_vars=genre_columns,
                             var_name='Genre', value_name='Is_Genre')

filtered_df = melted_df[melted_df['Is_Genre'] == 1]

# Grouping by certificate and genre to calculate the average rating
cert_genre_relation = filtered_df.groupby(['certificate', 'Genre'])['rating'].mean().reset_index()

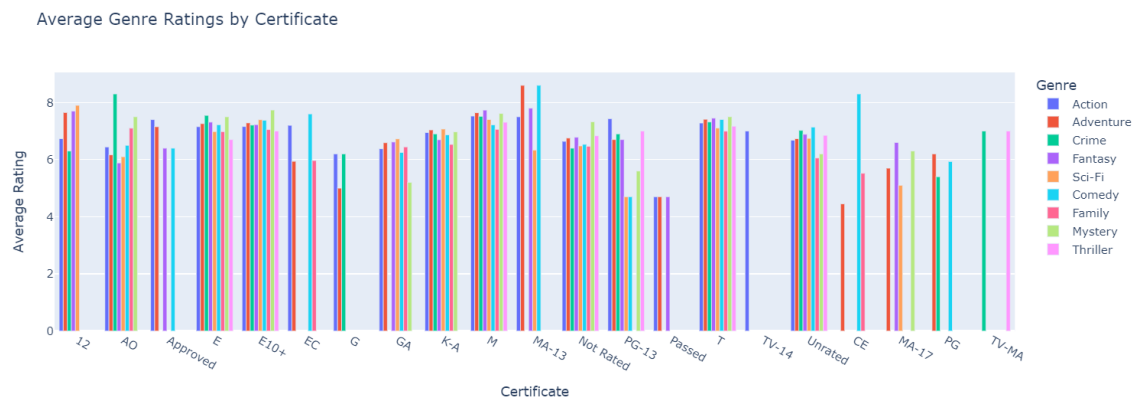
fig = px.bar(cert_genre_relation, x='certificate', y='rating', color='Genre',
              title='Average Genre Ratings by Certificate',
              labels={'certificate': 'Certificate', 'rating': 'Average Rating'},
              barmode='group')

fig.update_layout(xaxis_title='Certificate',
                  yaxis_title='Average Rating',
                  legend_title='Genre')

fig.show()
```

[11] ✓ 0.0s Python

...



There are clear variations in average ratings across different genres. For example, “Adventure” genre consistently has higher average ratings, while “Family” genre generally has lower average ratings.

While some genres consistently have high or low average ratings, there are occasional outliers, like “Comedy” having a very high average rating for the “CE” certificate.

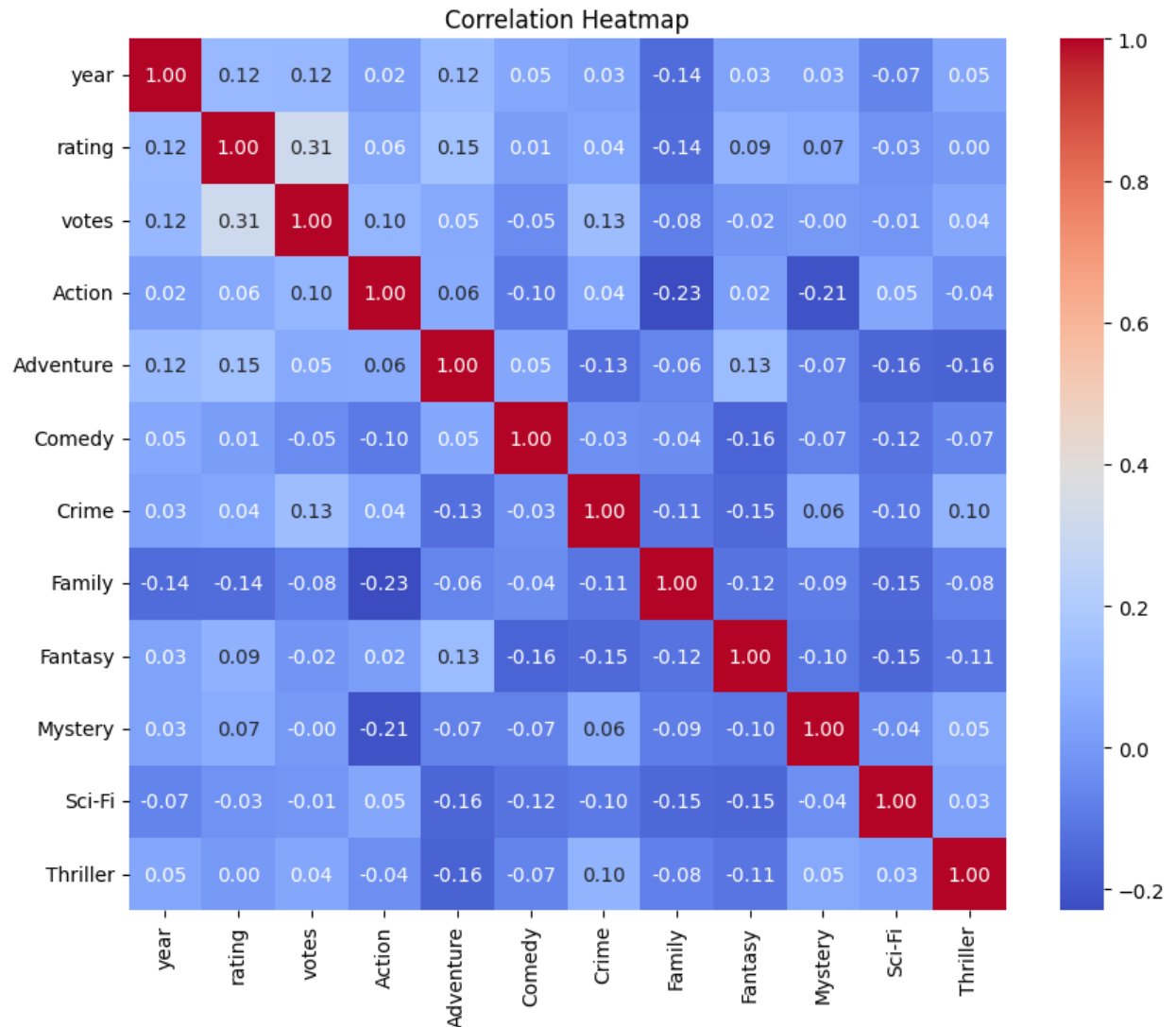
Correlation Heatmap

```
numeric_df = df_cleaned.select_dtypes(include=[np.number])

plt.figure(figsize=(10, 8))
correlation_matrix = numeric_df.corr() # Calculating correlation on numeric DataFrame
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

[23] ✓ 0.2s Python

Resulting graph:



There's a positive correlation between rating and votes, meaning that games with higher ratings tend to have more votes.

There are also some moderate correlations between different genres. For example, Action and Adventure genres have a moderate positive correlation, suggesting that they share some similar characteristics. But still, most of the other correlations are weak or nonexistent.